

数据融合与智能分析实验报告 1-2

班级	物联网 2202	学号	223428010210
专业	物联网工程	姓名	陈梓欣
实验成绩			
指导教师	郭振洲	2025 年 6 月 17 日	

一、实验题目：Numpy 与 Pandas 基础应用

二、实验地点：机械 307-6、雨课堂

三、实验目的：

- 1 理解并掌握 Numpy 库函数的常用操作；
- 2 理解并掌握 Pandas 库函数、DataFrame 对象的的常用操作；
- 3 理解并掌握数据缺失值的处理和分类统计的功能。

四、实验内容（完成题目代码及运行结果，代码注释中补充个人信息，以示区别）

1. Python 基础

（1）两种温度转换程序

```
# 223428010210 陈梓欣
# 两种温度转换程序
val = input("请输入温度和温度类型符号(如: 32C,40F): ")
if val [-1] in ['C','c']:
    f = 1.8 * float(val [0:-1]) + 32
    #print("转换后的温度结果为: %.2fF"%f)
    print(f"转换后的温度结果为:{f}f")
elif val [-1] in ['F','f']:
    c = (float(val [0:-1]) - 32) / 1.8
    print("转换后的温度结果为: %.2fC"%c)
else:
    print("输入错误! ")
```

请输入温度和温度类型符号(如: 32C, 40F): 32C
转换后的温度结果为:89.6f

(2) 字符串遍历

```
# 223428010210 陈梓欣
#字符串基本操作
str1 = 'abcdef'
str2 = "abcdefg"
v for x in str1:
    print(x)
```

a
b
c
d
e
f

(3) 字符串编码与译码

```
# 223428010210 陈梓欣
#字符串编码
s = '中国'
bs_utf = s.encode('utf-8')
print(bs_utf)
bs_gbk = s.encode('gbk')
print(bs_gbk)
```

b' \xe4\x b8\xad\xe5\x9b\xbd'
b' \xd6\xd0\xb9\xfa'

```
# 223428010210 陈梓欣
#字符串解码
print(bs_utf.decode('utf-8'))
print(bs_gbk.decode('gbk'))
```

中国
中国

(4) 字符串索引

```
# 223428010210 陈梓欣
#字符串索引
s1 = 'python最流行'
index = 0
v while index < len(s1):
    #print('%d:%s'%(index,s1[index]))
    print(f'{index}:{s1[index]}')
    index = index +1
```

0:p
1:y
2:t
3:h
4:o
5:n
6:最
7:流
8:行

(5) 字符串子串操作

```
# 223428010210 陈梓欣
# 子串的截取, 注意范围的右端开放的, 取不到
s1 = 'python最流行'
print(s1[1:5])
```

ytho

(6) 列表遍历

```
# 223428010210 陈梓欣
# 列表
l1 = [1,2,3,'a','b','d']
index = 0
while index < len(l1):
    print(f'{index}:{l1[index]}')
    index = index + 1
```

0:1
1:2
2:3
3:a
4:b
5:d

(7) 列表增加, 插入和删除

```
# 223428010210 陈梓欣
# 列表插入, 增加
l1 = [1,2,3,'a','b','c']
l1.append('d')
print(l1)
l1.insert(3,4)
print(l1)
```

[1, 2, 3, 'a', 'b', 'c', 'd']
[1, 2, 3, 4, 'a', 'b', 'c', 'd']

```
# 223428010210 陈梓欣
# 列表删除, remove 按内容删除
l3 = [1, 2, 3, 4, 'a', 'b', 'c', 'b']
l3.remove('b')
print(l3)
```

[1, 2, 3, 4, 'a', 'c', 'b']

```
# 223428010210 陈梓欣
# 列表删除, pop 需要输入元素的索引
l3 = [1, 2, 3, 4, 'a', 'b', 'c', 'b']
l3.pop(5)
print(l3)
```

[1, 2, 3, 4, 'a', 'c', 'b']

(8) 字典对象的遍历

```
# 223428010210 陈梓欣
# 字典的遍历
dic = {1:'zhangsan',2:'lisi',3:'wangwu','abc':'bcd'}
v for key in dic:
    print(f'{key}:{dic[key]}')
```

```
1:zhangsan
2:lisi
3:wangwu
abc:bcd
```

(9) 文件读取操作

```
# 223428010210 陈梓欣
# 文件读取 mode
f = open("./python简介-gb.txt", mode="r")
content = f.read()
print(content)
```

Python由荷兰国家数学与计算机科学研究中心的吉多·范罗苏姆于1990年代初设计，作为一门叫做ABC语言的替代品。 [1]Python提供了高效的高级数据结构，还能简单有效地面向对象编程。Python语法和动态类型，以及解释型语言的本质，使它成为多数平台上写脚本和快速开发应用的编程语言， [2]随着版本的不断更新和语言新功能的添加，逐渐被用于独立的、大型项目的开发。 [3] Python在各个编程语言中比较适合新手学习，Python解释器易于扩展，可以使用C、C++或其他可以通过C调用的语言扩展新的功能和数据类型。 [4]Python也可用于可定制化软件中的扩展程序语言。Python丰富的标准库，提供了适用于各个主要系统平台的源码或机器码。

```
# 223428010210 陈梓欣
# 文件读取 open(文件路径, mode=操作模式, encoding=编码方式)
f = open("./python简介.txt", mode="r", encoding="utf-8")
content = f.read()
print(content)
```

Python由荷兰国家数学与计算机科学研究中心的吉多·范罗苏姆于1990年代初设计，作为一门叫做ABC语言的替代品。 [1]Python提供了高效的高级数据结构，还能简单有效地面向对象编程。Python语法和动态类型，以及解释型语言的本质，使它成为多数平台上写脚本和快速开发应用的编程语言， [2]随着版本的不断更新和语言新功能的添加，逐渐被用于独立的、大型项目的开发。 [3] Python在各个编程语言中比较适合新手学习，Python解释器易于扩展，可以使用C、C++或其他可以通过C调用的语言扩展新的功能和数据类型。 [4]Python也可用于可定制化软件中的扩展程序语言。Python丰富的标准库，提供了适用于各个主要系统平台的源码或机器码。

(10) 文件修改操作

```
# 223428010210 陈梓欣
# 文件读取 open(文件路径, mode=操作模式, encoding=编码方式)
f = open("./射雕三部曲.txt", mode="r+", encoding = 'utf-8')
content = f.read()
print(content)
```

```
射雕英雄传
神雕侠侣
倚天屠龙记
```

```
# 223428010210 陈梓欣
#文件增加内容 mode="w+" 直接覆盖且支持写和读
v with open("./射雕三部曲.txt", mode="w+", encoding="utf-8") as f1:
    f1.write("鹿鼎记")
    f1.seek(0)
    content = f1.read()
    print(content)
```

```
鹿鼎记
```

```
# 223428010210 陈梓欣
# 文件增加内容 mode="w+" 直接覆盖且支持写和读
f1 = open("./射雕三部曲.txt", mode="w+", encoding="utf-8")
f1.write("鹿鼎记")
f1.seek(0) # 写完后回到文件开头准备读
content = f1.read()
print(content)
f1.close()
```

鹿鼎记

```
# 223428010210 陈梓欣
with open("./射雕三部曲.txt", mode="a+", encoding="utf-8") as f1:
    f1.write("\n鹿鼎记")
    f1.seek(0)
    content = f1.read()
    print(content)
```

射雕英雄传
神雕侠侣
倚天屠龙记
鹿鼎记

```
# 223428010210 陈梓欣
f1 = open("./射雕三部曲.txt", mode="a+", encoding="utf-8")
f1.write("\n鹿鼎记")
f1.seek(0) # 写完后回到文件开头准备读
content = f1.read()
print(content)
f1.close()
```

射雕英雄传
神雕侠侣
倚天屠龙记
鹿鼎记

2. Numpy 基础

(1) 列表对象初始化数组

```
# 223428010210 陈梓欣
# Numpy 基础
# 用列表初始化数组
import numpy as np
arr = np.array([1,2,3])
arr
```

array([1, 2, 3])

(2) 一维行向量转换列向量

```
# 223428010210 陈梓欣
# 一维行向量转换一维列向量
arr2 = arr1[:,np.newaxis]
print(arr2.shape)
arr2
```

(3, 1)
array([[1],
[2],
[3]])

(3) 读取图片为一个数组

```
# 223428010210 陈梓欣
import matplotlib.pyplot as plt
img_arr = plt.imread('./熊出没.jpg')
plt.imshow(img_arr)
```

<matplotlib.image.AxesImage at 0x201761e4e30>



```
# 223428010210 陈梓欣
img_arr.shape
```

(817, 676, 3)

(4) 常用基础矩阵，全 1，全 0 和单位阵

```
# 223428010210 陈梓欣
# 全1矩阵
a=np.ones(shape=(3,4))
# 全0矩阵
b=np.zeros(shape=(3,4))
# 单位阵
c=np.identity(3)
print(f"a={a}")
print(f"b={b}")
print(f"c={c}")
```

```
a=[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
b=[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
c=[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

(5) 一维数组 linspace()与 range()的区别

```
# 223428010210 陈梓欣
# linspace(起点, 终点, num=总的分割点数, 包含端点=分割份数+1)
list1 = np.linspace(0, 100, num=21)
# arange(起点, 终点(取不到), step=公差)
list2 = np.arange(0, 100, step=5)
print(f"list1 = {list1}")
print(f"list2 = {list2}")
# list1 能取到100, 是浮点数; list2 到95, 整数。性能好一些。
```

```
list1 = [ 0.  5. 10. 15. 20. 25. 30. 35. 40. 45. 50. 55. 60. 65.
 70. 75. 80. 85. 90. 95. 100.]
list2 = [ 0  5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95]
```

3. 自定义一个 5*6 的二维数组 matrix, 分别实现如下操作:

(1) 读取 matrix 的形状 维度, 元素总数, 数据类型等信息;

```
# 223428010210 陈梓欣
import numpy as np
matrix = np.random.randint(0, 100, size=(5, 6))
matrix
```

```
array([[64, 17, 11, 19, 87, 46],
       [85, 29, 63,  9, 98, 36],
       [13, 24, 16, 93, 54, 97],
       [39, 19, 81, 11, 91, 40],
       [ 8, 80, 22, 26, 21, 95]])
```

```
# 223428010210 陈梓欣
# 形状 维度, 元素总数, 数据类型
print(matrix.shape)
print(matrix.ndim)
print(matrix.size)
print(matrix.dtype)
```

```
(5, 6)
2
30
int32
```

(2) 对 matrix 进行截取和切片操作, 返回第 1 行和第 1 列向量;

```
# 223428010210 陈梓欣
# 取第一行
matrix[0]
# arr2[0, :]
```

```
array([64, 17, 11, 19, 87, 46])
```

```
# 223428010210 陈梓欣
# 取第一列
matrix[:, 0]
```

```
array([64, 85, 13, 39,  8])
```

(3) 通过::-1 的对二维数组 matrix 元素进行行翻转和列翻转;

```
# 223428010210 陈梓欣
print(matrix)
print("行翻转:",matrix[::-1])
print("列翻转:",matrix[:,::-1])
```

```
[[64 17 11 19 87 46]
 [85 29 63  9 98 36]
 [13 24 16 93 54 97]
 [39 19 81 11 91 40]
 [ 8 80 22 26 21 95]]
行翻转: [[ 8 80 22 26 21 95]
 [39 19 81 11 91 40]
 [13 24 16 93 54 97]
 [85 29 63  9 98 36]
 [64 17 11 19 87 46]]
列翻转: [[46 87 19 11 17 64]
 [36 98  9 63 29 85]
 [97 54 93 16 24 13]
 [40 91 11 81 19 39]
 [95 21 26 22 80  8]]
```

(4) 对二维数组进行行统计和列统计;

```
# 223428010210 陈梓欣
# 行统计
row_sum = np.sum(matrix, axis=1)      # 每一行的和
row_mean = np.mean(matrix, axis=1)    # 每一行的平均值
row_max = np.max(matrix, axis=1)      # 每一行的最大值
row_min = np.min(matrix, axis=1)      # 每一行的最小值

# 列统计
col_sum = np.sum(matrix, axis=0)      # 每一列的和
col_mean = np.mean(matrix, axis=0)    # 每一列的平均值
col_max = np.max(matrix, axis=0)      # 每一列的最大值
col_min = np.min(matrix, axis=0)      # 每一列的最小值

# 打印结果
print("\n【行统计】")
for i in range(matrix.shape[0]):
    print(f"第{i+1}行 → 和: {row_sum[i]}, 平均值: {row_mean[i]:.2f}, 最大值: {row_max[i]}, 最小值: {row_min[i]}")

print("\n【列统计】")
for j in range(matrix.shape[1]):
    print(f"第{j+1}列 → 和: {col_sum[j]}, 平均值: {col_mean[j]:.2f}, 最大值: {col_max[j]}, 最小值: {col_min[j]}")
```

【行统计】

第1行 → 和: 244, 平均值: 40.67, 最大值: 87, 最小值: 11
第2行 → 和: 320, 平均值: 53.33, 最大值: 98, 最小值: 9
第3行 → 和: 297, 平均值: 49.50, 最大值: 97, 最小值: 13
第4行 → 和: 281, 平均值: 46.83, 最大值: 91, 最小值: 11
第5行 → 和: 252, 平均值: 42.00, 最大值: 95, 最小值: 8

【列统计】

第1列 → 和: 209, 平均值: 41.80, 最大值: 85, 最小值: 8
第2列 → 和: 169, 平均值: 33.80, 最大值: 80, 最小值: 17
第3列 → 和: 193, 平均值: 38.60, 最大值: 81, 最小值: 11
第4列 → 和: 158, 平均值: 31.60, 最大值: 93, 最小值: 9
第5列 → 和: 351, 平均值: 70.20, 最大值: 98, 最小值: 21
第6列 → 和: 314, 平均值: 62.80, 最大值: 97, 最小值: 36

4. 读取一个图片并完成下列操作：

(1) 对图片进行水平翻转，并用级联方法与原图进行水平拼接，并显示出来；

```
# 223428010210 陈梓欣
# 图像水平翻转拼接
img_arr3 = img_arr[:,::-1,:]
img_out1 = np.concatenate((img_arr,img_arr3), axis = 1)
plt.imshow(img_out1)
```

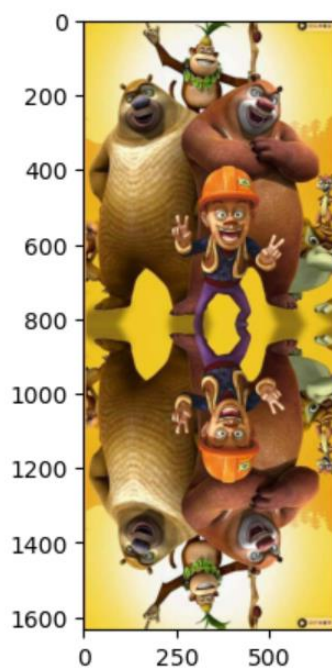
<matplotlib.image.AxesImage at 0x2017749d1f0>



(2) 对图片进行垂直翻转，并用级联方法与原图进行垂直拼接，并显示出来；

```
# 223428010210 陈梓欣
# 图像垂直翻转拼接
img_arr2 = img_arr[::-1,:]
img_out = np.concatenate((img_arr,img_arr2), axis = 0)
plt.imshow(img_out)
```

<matplotlib.image.AxesImage at 0x201775f01d0>



5. 随机生成两个 3*3 的整数矩阵 A 和 B，分别实现如下操作：

(1) 求 $C=A*B$, C 中元素为矩阵 A 和 B 对应元素相乘；

```
# 223428010210 陈梓欣
C = A * B # 或 np.multiply(A, B)
print("C = A * B (对应元素相乘): \n", C)
```

C = A * B (对应元素相乘) :

```
[[ 189  455 2340]
 [ 702  416 5810]
 [1537  581  588]]
```

(2) 求 $D=AB$, D 为 A 与 B 矩阵乘法；

```
# 223428010210 陈梓欣
D = np.dot(A, B) # 或 A @ B
print("D = AB (矩阵乘法): \n", D)
```

D = AB (矩阵乘法) :

```
[[ 6481  3929 12196]
 [ 5842  7370  3488]
 [ 5659  8391  3688]]
```

(3) 求矩阵 A 的逆矩阵，并验证逆矩阵的性质。

```
# 223428010210 陈梓欣
# 判断是否可逆 (行列式不为0)
det_A = np.linalg.det(A)
print("A 的行列式值: ", det_A)

if det_A != 0:
    A_inv = np.linalg.inv(A)
    print("A 的逆矩阵: \n", A_inv)

    # 验证: A × A_inv ≈ 单位矩阵
    identity = np.dot(A, A_inv)
    print("identity: \n", identity)
    print("验证 A × A-1 是否为单位矩阵: \n", np.round(identity, 2))
else:
    print("矩阵 A 不可逆 (行列式为 0)")
```

A 的行列式值: 209785.99999999997

A 的逆矩阵:

```
[[ 0.00937622 -0.04164244  0.03278102]
 [ 0.00540074  0.02583585 -0.02331423]
 [-0.00316036  0.01047734  0.00216888]]
```

identity:

```
[[ 1.00000000e+00  1.87350135e-16 -1.37043155e-16]
 [ 7.28583860e-17  1.00000000e+00 -7.02563008e-17]
 [ 1.04083409e-16 -6.24500451e-17  1.00000000e+00]]
```

验证 $A \times A^{-1}$ 是否为单位矩阵:

```
[[ 1.  0. -0.]
 [ 0.  1. -0.]
 [ 0. -0.  1.]]
```

(4) 对 A 和 B 实现列合并和行合并

```
# 223428010210 陈梓欣
# 行合并 (垂直方向) → 上下拼接
row_combined = np.vstack((A, B))
print("行合并: \n", row_combined)

# 列合并 (水平方向) → 左右拼接
col_combined = np.hstack((A, B))
print("列合并: \n", col_combined)
```

行合并:

```
[[63 91 26]
 [13 26 83]
 [29  7 98]
 [ 3  5 90]
 [54 16 70]
 [53 83  6]]
```

列合并:

```
[[63 91 26  3  5 90]
 [13 26 83 54 16 70]
 [29  7 98 53 83  6]]
```

五、实验中遇到的问题及解决方法

问题：在进行“文件修改”操作时，无论是“直接覆盖”操作还是“追加写”操作。

解决方法：

- (1) 对于直接覆盖操作，“w”模式，是只写模式，不允许读取操作。可将其修改为用“w+”模式，可实现读写均可；或者写完之后重新以读模式打开读取。
- (2) 对于追加写操作，同样用的是“a”模式（追加写模式），只能写不能读。可将其修改为用“a+”模式，一步完成；或者用“a”模式追加写入，然后再读取内容。

六、实验总结及心得

通过本次实验，我深入学习并掌握了 Python 中 NumPy 与 Pandas 两大数据分析库的基本用法与核心功能。首先，在 NumPy 部分，我了解了如何使用 `np.array` 创建数组、如何通过 `np.sum`、`np.mean`、`np.dot` 等函数实现矩阵的基础运算，如对应元素相乘、矩阵乘法、转置和求逆等操作，这加深了我对线性代数知识在编程中的实际应用理解。

随后在 Pandas 部分，我学习了如何构建 `DataFrame` 对象，以及如何进行数据选取、分组统计、排序和描述性分析等常规操作。同时，我还掌握了对数据中缺失值的处理方法，包括 `dropna()` 删除缺失值和 `fillna()` 填充缺失值等函数的使用。这些功能为数据清洗与预处理提供了强大支持。

此外，分类汇总与统计功能让我学会了如何对不同类别的数据进行分析与对比，为后续更复杂的数据挖掘任务打下了基础。本次实验不仅提升了我对数据结构和数据处理工具的熟练度，也增强了我对数据分析流程的整体理解和实践能力，为今后进行实际数据分析项目提供了技术保障。